

CHAPTER 1: INTRODUCTION

1.1 BACKGROUND OF THE STUDY

In digital image processing, handheld and edge consumer devices face severe physical and hardware constraints that compromise sensor quality. Under low-light conditions, raw smartphone sensors capture fewer photons, introducing high-intensity thermal and electronic noise that degrades spatial clarity (Gonzalez & Woods, 2018). This noise is conventionally modeled as Additive White Gaussian Noise (AWGN) in the spatial domain (Gonzalez & Woods, 2018). While standard spatial denoising filters—such as bilateral, median, and anisotropic diffusion filters—are highly effective at suppressing high-frequency noise, they inevitably smooth out critical high-frequency structural edges and fine textures (Gonzalez & Woods, 2018). The loss of these structural boundaries degrades image sharpness and spatial resolution, presenting a major barrier to high-fidelity consumer photography and mobile-based computational imaging.

To preserve sharp edge transitions while filtering out noise, Compressive Sensing (CS) theory provides a powerful mathematical alternative. CS asserts that natural images exhibit approximate sparsity when projected onto appropriate transform domains, such as the Discrete Cosine Transform (DCT) or Discrete Wavelet Transform (DWT) (Candès & Wakin, 2008). This sparse representation allows the denoising task to be modeled as an optimization problem over an underdetermined linear system (Baraniuk, 2007). Standard convex relaxation methods, such as Basis Pursuit Denoising (BPDN) and LASSO, substitute the non-convex L_0 norm with a convex L_1 proxy (Tibshirani, 1996; Chen et al., 2001). While the L_1 -norm optimization ensures a convex landscape and a guaranteed global minimum, it requires computationally expensive algorithms—such as the Alternating Direction Method of Multipliers (ADMM)—to solve a large system of linear equations at each iteration (Boyd et al., 2011). This computational burden can increase processing latency and resource utilization on resource-constrained devices, motivating the investigation of computationally efficient sparse-recovery methods.

Recent advancements in military radar signal processing have successfully utilized high-speed, non-convex single-loop solvers to recover high-resolution Inverse Synthetic Aperture Radar (ISAR) images (Feng et al., 2021). Algorithms such as the Improved Single-Loop Smoothed L_0 (ISSL $_0$) and Fast Approximate L_0 (FAL $_0$) optimize the reconstruction path by approximating the discontinuous, NP-hard L_0 norm using smooth, continuous Gaussian functions (Mohimani et al., 2009; Jin et al., 2019). This approximation allows the optimization problem to be solved using gradient descent or modified Newton methods within a single, unified loop, bypassing the expensive nested "inner-outer" loops characteristic of traditional continuation frameworks (Mohimani et al., 2009; Feng et al., 2021).

However, transferring these single-loop radar algorithms to natural 2D image denoising under compressive sensing constraints introduces severe mathematical and physical limitations:

The DC-Scrambling Problem: In military ISAR imaging, the target is strictly sparse (a few reflective metal points against a silent, empty sky background) (Feng et al., 2021). Natural 2D images are generally compressible or approximately sparse in transform domains such as DCT or wavelets, with a small number of dominant coefficients and many lower-amplitude coefficients

contributing texture and gradual structure. Standard random compressive sensing projections scramble this critical, non-sparse DC coefficient with high-frequency AC details. Random sensing may increase reconstruction error in the DC component because the local mean is represented indirectly through compressed measurements. The contribution of this effect is evaluated through a controlled comparison with a DC-preserving sensing configuration.

Optimization Instability: The non-convex landscape of the smoothed L_0 approximation is highly susceptible to local minima in underdetermined, noisy settings. Without a mathematically rigorous line-search and adaptive regularization framework, gradient descent steps can easily overshoot the optimal coefficients, causing divergence or false convergence stalls.

This study directly addresses these challenges by developing a mathematically consistent, single-loop hybrid sparse recovery solver. The proposed solver incorporates an unconstrained regularized objective function, a stable Singular Value Decomposition (SVD)-based pseudoinverse initialization, and a mathematically rigorous Armijo backtracking line search with corrected failure-path continuation logic. Furthermore, to reduce boundary discontinuities and block-related artifacts, the solver is integrated into an overlapping patch-reconstruction pipeline utilizing a dense stride and a normalized 2D Hamming Window weighting scheme. A distinct DC-preserving compressive sensing experiment is also established, offering a practical speed-efficiency trade-off tailored for low-power mobile edge processors.

1.2 REVIEW OF RELATED LITERATURE (THEMATIC SYNTHESIS)

To anchor the proposed hybrid solver, the related literature is synthesized across four distinct mathematical and technological domains:

1.2.1 Convex Relaxation and Greedy Baselines in Compressive Sensing

The mathematical core of sparse signal recovery is governed by the underdetermined linear system: $y = Az + e$ where $y \in \mathbb{R}^M$ represents the compressed measurement vector, $A \in \mathbb{R}^{M \times N}$ is the underdetermined sensing matrix (with $M < N$), $z \in \mathbb{R}^N$ represents the sparse transform-domain coefficients, and e is measurement noise (Candès & Tao, 2005; Donoho & Elad, 2003).

To solve this underdetermined system, traditional convex relaxation methods replace the discontinuous L_0 norm with the convex L_1 norm (Tibshirani, 1996). Under the LASSO framework, the Alternating Direction Method of Multipliers (ADMM) decomposes the global optimization problem into smaller, decoupled subproblems (Boyd et al., 2011). Although ADMM guarantees stable convergence, it requires the repeated solution of a linear system involving $(A^T A + \rho I)$. When the system matrix is fixed, its factorization can be precomputed and reused across iterations (Boyd et al., 2011). The repeated solution of the resulting linear system contributes substantial computational cost, particularly when the system is large or when many overlapping image patches must be processed.

Conversely, greedy algorithms like Orthogonal Matching Pursuit (OMP) avoid joint optimization by sequentially identifying the dictionary columns (atoms) that exhibit the highest correlation with the current residual (Tropp & Gilbert, 2007). In each step, OMP projects the residual onto the selected support set and solves a least-squares problem (Needell & Vershynin, 2009). While

OMP is highly effective at isolating dominant, high-contrast edges in synthetic blocks, its sequential projections are computationally heavy when scaling to thousands of overlapping image patches on low-power mobile CPUs (Davis, 1997; Chen et al., 2001).

1.2.2 Non-Convex Smoothed L_0 Continuation and Single-Loop Architectures

To bypass both the expensive matrix inversions of ADMM and the sequential dictionary searches of OMP, Mohimani et al. (2009) introduced the Smoothed L_0 (SL_0) algorithm. SL_0 approximates the discrete L_0 norm using a continuous, parameterized Gaussian function: $|z|_0 \approx N - \sum e^{-(z_i^2 / 2\sigma^2)}$ where the continuation parameter σ dictates the smoothness of the approximation (Mohimani et al., 2009).

Historically, SL_0 was implemented as a double-loop algorithm: the outer loop slowly decays σ to trace a path toward the sharp, non-convex L_0 norm, while the inner loop executes multiple gradient descent steps to reach convergence for that fixed σ (Mohimani et al., 2009). To eliminate the redundant computational cycles of the inner loop, single-loop architectures—such as the Improved Single-Loop Smoothed L_0 (IS SL_0) and Combined Regularized Smooth L_0 (CRe SL_0)—were developed for high-resolution radar imaging (Feng et al., 2021; Wang et al., 2019). By updating the continuation parameter σ geometrically at every single iteration, these solvers achieve substantial speedups (Wang et al., 2019; Quan et al., 2024).

However, applying these radar solvers to natural images introduces severe optimization instabilities, as natural signals contain small, non-zero AC coefficients that are aggressively driven to zero by the L_0 penalty, causing representation loss and spatial blurring (Wimalajeewa et al., 2012; Babacan et al., 2010).

1.2.3 Active-Set Methods and Support Pruning (FAL $_0$)

To further reduce the mathematical overhead of iterative solvers, active-set methods restrict gradient evaluations to a subset of estimated non-zero coefficients. The Fast Approximate L_0 (FAL $_0$) algorithm utilizes a positive definite matrix to stabilize convergence and incorporates a "Zero-Element Neglect" strategy (Jin et al., 2019). By identifying inactive coefficients (whose magnitudes fall below a threshold proportional to σ) and excluding them from subsequent gradient updates, the solver dramatically reduces floating-point operations (FLOPs) (Jin et al., 2019).

Despite these computational savings, hard active-set methods suffer from permanent coefficient deletion: once a coefficient is falsely flagged as zero and pruned, its gradient is ignored, and it can never be recovered (Jin et al., 2019). This leads to severe visual degradation in natural images, where soft textures and low-amplitude edges are permanently wiped out.

The ASL-SR-DPT implementation mitigates this limitation by periodically reopening the active support, allowing previously excluded coefficients to re-enter subsequent optimization iterations.

1.2.4 Spatial Image Reconstruction and Overlapping Patch Aggregation

In digital image denoising, processing images block-by-block is standard practice to preserve local spatial statistics (Gonzalez & Woods, 2018). However, independent reconstruction of

adjacent ⁸ pixel blocks causes severe spatial discontinuity at the boundaries, resulting in highly visible grid and block-blocking artifacts (Gonzalez & Woods, 2018).

To mitigate this, state-of-the-art pipelines decompose images using an overlapping stride and aggregate the overlapping patches using weighted windowing (Gonzalez & Woods, 2018). Multiplying overlapping spatial patches by a 2D Hamming Window and dividing by the accumulated spatial weight canvas ensures smooth, continuous boundaries (Gonzalez & Woods, 2018; Li & Qi, 2015). This digital signal processing standard is highly compatible with sparse-recovery solvers, helping to reduce boundary discontinuities and block-related artifacts and recover visual quality.

1.3 THEORETICAL FRAMEWORK

The theoretical foundation of this study is anchored in non-convex continuous optimization and compressive sampling theory, synthesizing the concepts of graduated continuation, sufficient descent, and active-set support restriction.

The theoretical framework rests on three foundational pillars. First, Compressive Sampling Theory (Candès & Wakin, 2008) models underdetermined recovery as $y = Az + e$, establishing the information-theoretic limits of random projections. Second, Graduated Non-Convexity (Mohimani et al., 2009) approximates the discrete L_0 norm using continuous Gaussian functions and employs a continuation parameter (σ) to smooth local minima. Finally, Armijo-type sufficient descent provides a principled acceptance criterion for candidate steps for the current objective and active support. It does not by itself establish global convergence of the complete nonconvex continuation algorithm. Building on these principles, the proposed hybrid model defines the objective function as $\min_z 0.5 * \|Az - y\|_2^2 - \lambda * \sum (e^{-(z_i^2 / 2\sigma^2)})$. This model incorporates several key components: SVD-based Moore-Penrose pseudoinverse initialization ($P_{init} = A^\dagger$), single-loop geometric continuation with σ updating per iteration, active-set zero-element neglect with periodic support re-evaluation, and Armijo backtracking line search with corrected failure-path logic.

1.3.1 The Unconstrained Regularized Objective Function

Rather than forcing exact linear projections ($Az = y$), which preserves noise in the reconstructed image, this study utilizes an unconstrained regularized model: $L(z) = \frac{1}{2}\|Az - y\|_2^2 - \lambda \sum e^{-(z_i^2 / 2\sigma^2)}$. This formulation balances the L_2 data fidelity term (which minimizes residual reconstruction error) with a negative Gaussian smoothed L_0 penalty (which drives coefficients to exact zeros to promote sparsity). The regularization parameter λ is treated as a controlled hyperparameter whose value is selected through a documented validation procedure.

1.3.2 The Unified Gradient

The search direction of the solver is guided by the analytically derived unified gradient of the unconstrained objective function: $\nabla L(z) = A^T(Az - y) + \lambda (z / \sigma^2) \odot e^{-(z^2 / 2\sigma^2)}$ where $\odot$ denotes the Hadamard element-wise product. For the current active support, the analytical gradient provides the steepest descent direction with respect to the active variables.

1.3.3 SVD-Based Minimum-Norm Initialization

At initialization ($k = 0$), the initial coefficient vector is set as the minimum-norm least-squares estimate: $z^{(0)} = P_{\text{init}}$. For each fixed sensing-matrix configuration, the Moore-Penrose pseudoinverse ($P_{\text{init}} = A^\dagger = V\Sigma^\dagger U^T$) is computed once and reused for patch initialization. This precomputation ensures a highly stable starting point close to the true coefficients, avoiding repeated pseudoinverse construction for every image or experiment.

1.3.4 Armijo Backtracking and Corrected Failure-Path Logic

To support numerical stability and descent, the step size μ is governed by the Armijo condition (Nocedal & Wright, 2006): $L(z + \mu d) \leq L(z) + c \cdot \mu \cdot \nabla L(z)^T d$ where the search direction $d = -\nabla L(z)$, $c = 10^{-4}$ is the Armijo descent constant, and the step decay factor $\beta = 0.5$.

When backtracking fails, the solver executes a corrected failure-path routine to reduce the risk of unstable step updates and preserve the current optimization state:

It retains the coefficient vector: $z_{\text{new}} = z$.

It decays the persistent step size: $\mu \leftarrow \mu \times 0.5$.

It skips the continuation decay of σ , preserving the current objective landscape.

It bypasses the convergence relative-change calculation, eliminating false-convergence loops.

1.3.5 Zero-Element Neglect with Periodic Support Re-evaluation

To slash floating-point complexity, the solver restricts gradient evaluations to the active set: $\text{active_indices} = |z_i| > 10^{-5} \sigma$. Subsequent matrix-vector operations are performed strictly over the active sub-matrix A_{active} and sub-vector z_{active} . To prevent the permanent deletion of critical image details, the solver implements a Periodic Support Re-evaluation schedule: every 3 iterations, the active mask is reset to all True values, allowing temporarily suppressed coefficients to re-enter the optimization space if their gradient changes.

1.4 CONCEPTUAL FRAMEWORK AND RESEARCH PARADIGM

The conceptual framework translates the theoretical optimization steps into an applied software architecture. The flow of information and processing stages is modeled using a standard Input-Process-Output (IPO) paradigm, aligning with the Software Development Life Cycle (SDLC) framework mandated by the university guidelines.

INPUT	PROCESS	OUTPUT
1. Grayscale 2D Image (BSD68 benchmark images)	1. Image Decomposition - Overlapping stride - DCT Coefficient projection	1. Reconstructed Spatial Denoised Image
2. Input Noise Levels ($\sigma_{\text{noise}} = 15, 25, 50$)	2. Solver Execution (ASL-SR-DPT)	2. Performance Analytics - Spatial PSNR (dB)

INPUT	PROCESS	OUTPUT
	- SVD-Based Pseudoinverse Initialization - Single-Loop Continuation - Armijo Backtracking - Zero-Element Neglect - Periodic Support Re-evaluation	- Spatial SSIM - Spatial MSE - Iterative Solve Latency (seconds) - Active-Support Ratio
3. Sensing Matrices - Random Compressive - DC-Preserving	3. Spatial Aggregation - 2D Hamming Window weights	3. Functional streams - STREAMLIT prototype web application
4. User Parameters - Nominal compression ratio of 0.6, corresponding to 38 measurements for 64 DCT coefficients. In DC-preserving mode, the DC coefficient is uncompressed, and only the 63 AC coefficients are processed.	-	-

1.4.1 Narrative of the Paradigm

Input Phase: The system ingests clean grayscale images from the BSD68 benchmark dataset.

Simulated Additive White Gaussian Noise is added at the selected noise levels

$\sigma_{\text{noise}} \in \{15, 25, 50\}$. The system also receives the selected sensing configuration, nominal compression ratio, patch size, and patch extraction stride. In DC-preserving mode, the DC coefficient is retained without compression, while the remaining 63 AC coefficients are processed compressively.

Process Phase: The image is decomposed into overlapping 8×8 patches using a stride of 2 pixels and projected into the DCT domain. For each fixed sensing-matrix configuration, the Moore-Penrose pseudoinverse is computed once and reused for patch initialization. The ASL-SR-DPT solver then performs single-loop continuation, analytically evaluates the active-support gradient, applies Armijo backtracking for step-size selection, and restricts computation using Zero-Element Neglect. The active support is periodically reopened to allow previously excluded coefficients to re-enter the optimization. Recovered coefficients are transformed back to the spatial domain using the 2D IDCT and aggregated using normalized 2D Hamming-window weights.

Output Phase: The system produces the fully reconstructed denoised spatial image. Performance is evaluated using spatial PSNR, SSIM, and MSE, together with iterative solve latency and active-support statistics. The numerical results are recorded for comparative analysis, while the verified processing pipeline is demonstrated through a Streamlit prototype.

1.5 STATEMENT OF THE PROBLEM

The primary objective of this study is to address the computational cost and reconstruction-quality challenges associated with sparse-recovery image denoising under resource-constrained computational settings. This is achieved by designing, validating, and prototyping a single-loop hybrid sparse recovery solver that evaluates the trade-off between reconstruction quality and computational cost.

Specifically, the study addresses the following research objectives:

Formulate the unconstrained regularized objective function and analytically derive its unified gradient to model noisy compressive measurements without forcing exact projections.

Design an Armijo-type backtracking line search and a graduated continuation schedule to dynamically tune the descent step size (μ) and prevent optimization divergence.

Develop an active-set support restriction scheme (Zero-Element Neglect) with periodic support re-evaluation to reduce floating-point operations (FLOPs) without permanently pruning vital image coefficients.

Integrate a stable precomputation framework utilizing the SVD-based Moore-Penrose pseudoinverse strictly for initial minimum-norm estimation.

Establish an overlapping patch-reconstruction pipeline utilizing a dense extraction stride and a normalized 2D Hamming Window weighting scheme to reduce boundary discontinuities and block-related artifacts.

Develop a functional prototype web application based on an Agile Software Development Life Cycle (SDLC) model to transition the simulated mathematical solver into a practical, interactive computational tool.

1.6 SIGNIFICANCE OF THE STUDY

This study provides direct, practical, and theoretical benefits to several key stakeholders:

Mobile Software and Camera Application Developers: Provides a lightweight, mathematically stable, and computationally evaluated sparse recovery engine that can be compiled for low-power mobile operating systems whose computational characteristics may inform future deployment on resource-constrained mobile or edge devices.

Smartphone Consumers: Provides a potential computational framework for denoised image reconstruction under low-light conditions, with future application in resource-constrained imaging systems.

Academic Scholars and Computational Mathematicians: Contributes an experimental analysis of the behavior and computational trade-offs of non-convex smoothed-L0 sparse recovery when

applied to approximately sparse 2D image patches by establishing the mathematical boundary conditions, convergence failures, and spatial reconstruction limits of non-convex smoothed L_0 continuation methods when applied to approximately sparse 2D signals.

Future Researchers: Establishes a highly documented, scientifically honest benchmarking framework (utilizing robust OMP and Cholesky-factorized ADMM-Lasso controls) that serves as a baseline for subsequent edge-computing and optimization studies.

1.7 SCOPE AND LIMITATION OF THE STUDY

This study is strictly delimited to an in-silico software-based numerical simulation conducted in Python 3.12 within a closed desktop computing sandbox. The research design does not involve compiling code onto physical mobile hardware, such as ARM processors or DSPs, or evaluating physical hardware variables such as battery consumption, thermal throttling, or hardware-level memory behavior.

The final performance evaluation uses the complete set of 68 grayscale images in the BSD68 benchmark dataset. Synthetic 128×128 structural image blocks are used only for preliminary controlled tests of solver behavior and implementation correctness.

The image recovery is evaluated under an underdetermined, row-normalized Gaussian sensing model and a separate DC-preserving sensing configuration with a nominal compression ratio of 0.6, corresponding to 38 measurements for 64 DCT coefficients. In the DC-preserving configuration, the DC coefficient is retained without compression, while the remaining 63 AC coefficients are processed compressively. The software engineering output is limited to a functional, browser-based Streamlit prototype application demonstrating the patch-level processing speed and reconstruction quality of the verified Python engine.

1.8 DEFINITION OF TERMS

To ensure conceptual clarity and compliance, the core technical terms are defined both conceptually (theoretical basis) and operationally (how they are implemented in this study):

Active Support (Active Set):

Conceptual Definition: The subset of transform-domain coefficients whose magnitudes exceed the active-support threshold, defined in this study as $|z_i| > 10^{-5} \sigma$ (Jin et al., 2019).

Operational Definition: The Boolean mask array in the ASL-SR-DPT solver (`active_indices = |z| > 10-5  $\sigma$`) that restricts gradient and matrix multiplications to non-zero coefficients.

Additive White Gaussian Noise (AWGN):

Conceptual Definition: A basic physical noise model in digital signal processing characterized by a constant spectral density and a Gaussian probability distribution of noise amplitudes (Gonzalez & Woods, 2018).

Operational Definition: The simulated AWGN is added to the clean spatial image at the selected noise levels $\sigma_{\text{noise}} \in \{15, 25, 50\}$.

Armijo Backtracking Line Search:

Conceptual Definition: An inexact line search method in numerical optimization used to determine an appropriate step size along a descent direction by checking the sufficient descent condition (Nocedal & Wright, 2006).

Operational Definition: The iterative loop in the ASL-SR-DPT solver that dynamically scales down the descent step size μ by a factor of 0.5 until the candidate cost is less than or equal to the current cost plus a fraction of the gradient step.

Compressive Sensing (Compressed Sensing):

Conceptual Definition: A signal processing paradigm asserting that sparse or approximately sparse signals can be fully reconstructed from highly undersampled linear measurements (Candès & Wakin, 2008).

Operational Definition: The simulation framework that projects 8×8 image patches using a row-normalized Gaussian sensing matrix under an underdetermined nominal compression ratio of 0.6, corresponding to 38 measurements for 64 DCT coefficients. In DC-preserving mode, the DC coefficient is uncompressed, and only the 63 AC coefficients are processed.

Graduated Continuation Schedule:

Conceptual Definition: An optimization strategy that solves a sequence of smooth, approximated objective functions, gradually deforming them toward the non-convex target function to avoid local minima (Mohimani et al., 2009).

Operational Definition: The geometric decay of the continuation parameter $\sigma = \max(\sigma \times 0.95, 0.01)$ executed at every iteration of the single-loop solver.

Peak Signal-to-Noise Ratio (PSNR):

Conceptual Definition: A logarithmic image-fidelity metric derived from the mean squared error between the reference image and the reconstructed image. Higher PSNR values indicate lower reconstruction error. Formula: $PSNR = 10 \log_{10}(MAX^2/MSE)$ (Gonzalez & Woods, 2018)

Operational Definition: The logarithmic spatial quality metric calculated strictly over the fully aggregated 2D reconstructed spatial block compared to the original clean spatial block.

Single-Loop Architecture:

Conceptual Definition: An optimization loop design that uses a single outer optimization and continuation loop with a bounded inner Armijo backtracking procedure to reduce computational complexity and redundant operations (Feng et al., 2021).

Operational Definition: The solver loop structure that updates the gradient, performs backtracking step-size updates, and decays the continuation parameter σ within a single for iteration in `range(max_iter)` block.

Zero-Element Neglect:

Conceptual Definition: An active-set optimization heuristic that excludes inactive, near-zero elements from vector calculations to reduce computational complexity and floating-point operations (Jin et al., 2019).

Operational Definition: The restriction of the ASL-SR-DPT solver's gradient evaluations strictly to the indices where the coefficients exceed the active support threshold.

Chapter 2

Research Design

This study employs a quantitative, experimental research design utilizing algorithmic simulation (in-silico sandbox experiments) to evaluate the performance of the ASL-SR-DPT. According to the Father Saturnino Urios University (FSUU) Institutional Student Research Manual, an experimental simulation design is highly appropriate for technological investigations where mathematical structures and algorithmic variables are systematically manipulated to observe their causal effects on system efficiency and output quality.

The mathematical and logical structure of the sparse recovery pipeline represents the independent variables, which are manipulated across three primary configurations:

Single-Loop continuation schedule (adapted from the ISSLO framework) versus classical double-loop architectures;

Dynamic Parameter Tuning (DPT), implemented through analytical gradient evaluation, Armijo-type backtracking, and controlled continuation of the smoothing parameter sigma, is evaluated against fixed-step and static-parameter approaches.

Active-Set Support Restriction (Zero-Element Neglect, adapted from FAL0 with periodic support re-evaluation) versus dense matrix-vector updates.

The dependent variables are divided into two primary evaluation domains:

Computational Efficiency: Measured quantitatively by iterative solve latency (seconds) and the average cardinality of active support sets (indicative of Floating-Point Operation, or FLOP, reductions).

Reconstruction Fidelity: Measured quantitatively by spatial Peak Signal-to-Noise Ratio (PSNR, in decibels), Structural Similarity Index (SSIM), and spatial Mean Squared Error (MSE) evaluated on the fully aggregated spatial images.

To ensure academic and scientific rigor, the proposed algorithm is benchmarked directly against two established, peer-reviewed sparse recovery baselines under identical operational constraints:

Orthogonal Matching Pursuit (OMP): A classic greedy algorithm utilizing column-normalized projections (Tropp & Gilbert, 2007).

LASSO-ADMM: A convex relaxation-based Alternating Direction Method of Multipliers solver using stable Cholesky factorization steps and primal-dual residual convergence tracking (Boyd et al., 2011).

By strictly manipulating the algorithms under identical noise levels and compressive measurement bounds, the research design provides a controlled basis for evaluating the reconstruction-quality and computational-cost trade-off among the proposed and baseline methods.

Research Environment, Population and Sample

This research does not take place in a physical geographic location or community environment. Instead, to simulate the strict hardware boundaries of a mobile processor while maintaining experimental consistency, the study is conducted within a controlled virtualized software environment running inside a virtualized sandbox hosted on an x86 developmental platform.

1. Hardware and Virtualization Environment (Simulated Mobile Profile)

To isolate execution latency and ensure timing and memory measurements are scientifically honest, reproducible, and with reduced variability from differences in guest resource allocation and software configuration, the simulation environment is built using virtualization technologies on a physical developmental computer.

Host Developmental Platform:

System Manufacturer & Model: LENOVO LOQ 15IRX9 (System Model 83DV).

Processor: 13th Gen Intel Core i5-13450HX (10 Physical Cores: 6 Performance Cores and 4 Efficient Cores, 16 Logical Processors, base clock speed running at 2.4 GHz). The host processor is documented to establish the hardware context of the virtualized benchmark environment.

Host OS: Microsoft Windows 11 Home Single Language (Version 10.0.26200 Build 26200).

Physical Memory: 16.0 GB DDR5 RAM.

Host Storage: SSD/NVMe drive (with Windows Hyper-V and Virtualization-Based Security (VBS) remaining active on the host to preserve realistic environment configurations).

Virtualization Hypervisor (Simulated Mobile Profile): The study utilizes VMware Workstation Pro, deploying a dedicated virtual machine running Ubuntu 24.04 LTS (UEFI firmware) as the Guest OS. This virtualized profile is designed strictly to cap and control hardware resources, reducing variability arising from differences in guest resource allocation and software configuration while ensuring consistent repeatability. The virtual hardware and virtualization engine are configured as follows:

Throttled Processor Allocation: Configured strictly to 1 Virtual CPU Package with 4 cores (4 vCPUs total), allocating only the physical resources that are effectively utilized by the underlying OS scheduler without causing excessive core-scheduling overhead.

Strict Memory Boundary: Configured with exactly 8.0 GB of RAM, providing a fixed memory allocation selected for the controlled benchmark environment for 2D image patch processing and BSD68 evaluation while ensuring no virtual memory paging or RAM starvation occurs inside the Guest OS.

Performance Optimization and Compatibility:

Host Storage: The VM virtual disk is hosted strictly on the physical SSD/NVMe-backed partition (60 GB+ allocated) to ensure stable disk I/O performance.

Disabled 3D Acceleration: Hardware 3D Graphics Acceleration is disabled (Off) inside VM settings to completely eliminate GPU-driven display-rendering distortions.

Virtualization Engine: Standard hardware virtualization remains enabled, while nested virtualization and virtual CPU performance counters (vPMCs) are disabled to prevent compatibility conflicts with Windows 11 Hyper-V and Virtualization-Based Security (VBS).

Peripheral Pruning: All non-essential virtual components—including USB controllers, Sound Cards, Printers, and CD/DVD drives—are completely disconnected and removed from the VM to eliminate background driver polling and interrupt request (IRQ) overhead.

Processor and Thread Affinity (Thread Locking): To completely eliminate multi-threaded scheduling overhead and ensure that NumPy and SciPy execute in a single-threaded execution context (mirroring standard mobile efficiency thread boundaries), all comparative benchmarks are executed after forcing the numerical backend libraries to a single execution thread inside the Guest OS terminal:

```
export OMP_NUM_THREADS=1
export OPENBLAS_NUM_THREADS=1
export MKL_NUM_THREADS=1
export NUMEXPR_NUM_THREADS=1
Python3 ASL-SR-DPT.py
```

This guarantees that the ASL-SR-DPT, OMP, and LASSO-ADMM solvers are measured on an identical, strictly single-threaded computational footing, removing multi-threaded OpenMP or BLAS compiler-optimization advantages. No GPU acceleration or specialized hardware co-processors are utilized, ensuring that all algorithms are evaluated under the same single-threaded numerical execution configuration.

To improve reproducibility, a clean VMware snapshot containing the operating system, Python environment, numerical libraries, and benchmark code is retained. The same snapshot and software configuration are used across controlled experimental runs.

Runtime is measured directly within the controlled VMware Workstation Pro environment. The measured latency represents execution performance under the specified virtualized x86 configuration and is not treated as a direct estimate of execution time on ARM-based mobile hardware. Mobile deployment is discussed as a potential application context rather than as a measured hardware result.

2. Guest Software Environment

Operating System: Ubuntu 24.04 LTS (running inside the VMware Workstation Pro guest partition, with all non-essential background system services and daemons disabled).

Programming Language: Python 3.12.3.

Core Numerical Libraries: NumPy 1.26.4 (compiled and linked against a single-threaded OpenBLAS library to prevent silent multi-core parallel matrix operations) and SciPy 1.12.0 (providing numerical linear-algebra routines used by the Cholesky-factorized LASSO-ADMM baseline.).

User Interface Framework: Streamlit 1.32.0 (powering the applied web prototype dashboard).

3. Dataset Environment (Population and Sample)

The evaluation dataset consists of the 68 grayscale images in the BSD68 benchmark. This dataset is widely recognized in digital image processing literature as a standard population for evaluating denoising and edge-preservation algorithms because it contains a rich variety of natural textures, structural boundary transitions, and uniform gradients.

The complete BSD68 dataset is used for final evaluation. Synthetic 128×128 structural blocks are used only for preliminary controlled tests of solver behavior and implementation correctness. These synthetic blocks are used only as controlled test cases for implementation verification because their known structure allows direct examination of reconstruction behavior.

Sampling Technique

Because this study evaluates a publicly available image benchmark rather than human participants, conventional participant-sampling procedures are not applicable. The complete BSD68 dataset is used for final evaluation. A separate validation subset of BSD68 images is used for preliminary testing, parameter selection, and ablation experiments.

The final benchmark does not exclude images based on texture complexity, spatial entropy, or edge content. This ensures that the reported results represent the performance of ASL-SR-DPT across the complete benchmark dataset rather than a selected subset of difficult or highly structured images.

For the validation and preliminary experiments, the following criteria are applied:

Format and Color Space: Images are processed as single-channel grayscale images. When source images are provided in another format, the luminance channel is extracted and converted to grayscale before processing.

Image and Patch Structure: Images are divided into overlapping 8×8 pixel patches. Preliminary controlled experiments may include patches containing edges, contours, textures, gradients, and relatively uniform regions to evaluate solver behavior across different local image structures.

Representativeness: The final evaluation includes all BSD68 images without purposive exclusion. This provides a broader assessment of reconstruction quality across natural image textures, structural boundaries, gradients, and varying local intensity distributions.

Ethical Consideration

The study uses publicly available benchmark images and does not involve human participants, animal subjects, or personally identifiable information. The researchers will follow the applicable institutional requirements for secondary-data computational studies.

Nonetheless, the researchers are committed to maintaining the highest standards of Intellectual Integrity, which is heavily evaluated under Criterion 4 of the FSUU Thesis Rubric:

Plagiarism and Originality: The manuscript and associated code documentation will comply with the current FSUU requirements for originality and academic integrity.

Mathematical and Algorithmic Attribution: Every mathematical mechanism adapted in this study is thoroughly cited. The single-loop continuation framework is credited to the ISSLO radar architecture (Feng et al., 2021), the active-set support restriction to the FALO framework (Jin et al., 2019), and the baseline configurations to their respective authors (Boyd et al., 2011; Tropp & Gilbert, 2007).

Scientific Honesty in Reporting: All empirical results, including cases in which the proposed method performs below a baseline, are reported transparently.

Research Instrument

For secondary-data software simulations, traditional physical instruments (such as survey questionnaires, physiological sensors, or physical measurement devices) are Not Applicable. Instead, the "instruments" in this study are composed entirely of mathematical operators, programmatic metrics, and software components compiled inside the Python environment. These computational instruments are defined as follows:

The Measurement Instrument (Sensing Matrix): An underdetermined row-normalized Gaussian sensing matrix $A \in \mathbb{R}^{(M \times N)}$ generated programmatically. The sensing matrix A is generated from independent Gaussian entries and each row is normalized to unit ℓ_2 norm. Row normalization does not imply $AA^T = I$. This matrix models the physical process of Compressive Sensing, mapping a high-dimensional coefficient vector $z \in \mathbb{R}^N$ (where $N = 64$ for an 8×8 patch) to a compressed measurement vector $y \in \mathbb{R}^M$ (where $M = 38$ for a compression ratio of 0.6).

The Transform Instrument (Discrete Cosine Transform): A 2D DCT orthonormal basis matrix $D \in \mathbb{R}^{(N \times N)}$ used to project spatial image patches into their approximately sparse representation.

The Analytical Instrument (The Python Solver Engine): The compiled Python script ASL-SR-DPT.py containing the exact mathematical formulations of the unconstrained regularized objective function, the unified analytical gradient, the Armijo backtracking line search, and the Zero-Element Neglect active-set restriction.

The Verification Instrument (The Prototype App): The interactive Streamlit dashboard streamlit_app.py used to visually execute, verify, and demonstrate the operational pipeline to the evaluation panel.

Data Gathering Procedure

To ensure that the experimental simulations can be fully understood, evaluated, and replicated by external computational scientists, the data gathering procedure is executed through a highly structured, six-phase programmatic pipeline.

Phase	Title	Activities
Phase 1	Environment Setup & Dataset Loading	Initialize local numpy random generators (seed=42); The complete BSD68 dataset is used for the benchmark evaluation. Preliminary solver verification, when required, is performed separately before the benchmark and is not included in the reported BSD68 performance results.

Phase	Title	Activities
Phase 2	Controlled Noise Injection	Inject AWGN across multiple noise levels with standard deviations ($\sigma = 15, 25, 50$) to evaluate performance and demonstrate robustness; generate corresponding noisy spatial images.
Phase 3	Compressive Sensing Modelling	Apply the selected sensing configuration, either standard row-normalized Gaussian sensing or DC-preserving sensing.
Phase 4	Algorithmic Solver Execution	For each fixed sensing-matrix configuration, the Moore-Penrose pseudoinverse ($P_{\text{init}} = A^\dagger$) is computed once; run OMP, LASSO-ADMM, and Proposed ASL-SR-DPT Hybrid Solver; track CPU solve latency and active support masks.
Phase 5	Spatial 2D Image Reconstruction	Reconstruct patches via 2D IDCT; aggregate overlapping pixels via 2D Hamming Window.
Phase 6	Analytical Logging & Extraction	Compute spatial PSNR, SSIM, and MSE on the fully aggregated reconstructed images. The numerical results, including execution time, iteration count, residual measures, and active-support statistics, are written to structured CSV files.

Detailed Procedural Execution:

Phase 1: Environment Setup and Dataset Loading. The software sandbox is initialized. To isolate resource metrics, background CPU tasks are terminated, and processor affinity is locked. The complete BSD68 dataset is used for the benchmark evaluation. Preliminary solver verification, when required, is performed separately before the benchmark and is not included in the reported BSD68 performance results.

Phase 2: Controlled Noise Injection. Additive White Gaussian Noise (AWGN) is independently generated at the selected noise levels

$$\sigma_{\text{noise}} \in \{15, 25, 50\}$$

and added to the clean image. The resulting noisy-image PSNR is recorded for each test image and used as the baseline for evaluating denoising improvement.

Phase 3: Compressive Sensing Modelling. The noisy image is decomposed into overlapping 8×8 pixel patches using a dense Stride of 2 pixels. For each patch, the measurement vector $y = A\theta + e$ is simulated. Under the DC-Preserving Experiment, the DC coefficient is extracted and

copied uncompressed directly to the reconstruction buffer, leaving only the 63 AC coefficients to be processed compressively. The model is defined as

$$\theta = [\theta_{DC}, \theta_{AC}]$$

$$y = A\theta$$

DC-preserving:

$$y_{AC} = A_{AC}\theta_{AC}$$

, where

$$\theta^{DC} = \theta_{noisy}^{DC}$$

The DC-preserving configuration retains the observed noisy DC coefficient without compression. It therefore preserves the local patch mean against compression-induced reconstruction error but does not independently remove noise from the DC component.

Phase 4: Algorithmic Solver Execution.

For each fixed sensing-matrix configuration, the Moore-Penrose pseudoinverse ($P_{init} = A^\dagger$) is computed once.

The initial "warm start" estimate $z^{(0)} = P_{init} \times y$ is computed for the patch.

The control algorithms (OMP and ADMM) and the Proposed ASL-SR-DPT Solver are executed inside the VMware Guest OS, with threading locked strictly to 1 thread via terminal environment variables.

During the ASL-SR-DPT iterations, the solver tracks active-support masks, executes the Armijo backtracking line search, and—if line search backtracking fails—triggers the corrected failure-path routine (retaining z , halving μ , and freezing σ decay) to reduce the risk of unstable step updates and preserve the current optimization state.

The high-precision timer logs the setup latency and the iterative solve latency separately.

Phase 5: Spatial 2D Image Reconstruction. The recovered sparse coefficient vectors are transformed back to the spatial domain via the 2D Inverse Discrete Cosine Transform (2D IDCT). To smooth block-boundary transitions, overlapping patches are aggregated using the normalized 2D Hamming Window weighting scheme.

Phase 6: Analytical Logging and Extraction. Compute spatial PSNR, SSIM, and MSE on the fully aggregated reconstructed images. The numerical results, including execution time, iteration count, residual measures, and active-support statistics, are written to structured CSV files.

Statistical Treatment

Because this is a controlled, reproducible algorithmic simulation with randomized sensing and noise trials, complex inferential statistics such as t-tests, ANOVA, or Chi-Square tests are not required for the primary analysis. Instead, the study uses descriptive statistics to summarize reconstruction quality, computational performance, and variability across randomized trials. Each randomized experiment is repeated across 50 independent trials using different random seeds for the sensing matrix and noise realization. The complete BSD68 image set remains fixed for the final benchmark evaluation. The arithmetic mean and standard deviation are calculated for PSNR, SSIM, MSE, and CPU solve latency to summarize the central tendency and variability of the observed results.

Arithmetic Mean (Average): The mean is computed as:

$$\bar{X} = \frac{1}{K} \sum_{k=1}^K X_k$$

where $K = 50$, and X_k represents the PSNR, SSIM, MSE, or CPU solve latency recorded during the k -th trial. All comparative results reported in the benchmarking tables represent these averaged values.

Standard Deviation: Calculated to verify the stability and reliability of the solvers, ensuring that the Proposed Solver does not experience chaotic convergence jumps across different randomized trials:

$$SD = \sqrt{\frac{1}{K-1} \sum_{k=1}^K (X_k - \bar{X})^2}$$

Relative Performance Ratios: Used to quantitatively measure the computational speedup of the proposed single-loop solver relative to the baseline methods. Since the study compares the proposed ASL-SR-DPT solver with both OMP and LASSO, speedup is calculated separately for each method:

$$\text{Speedup}_{\text{OMP}} = \frac{T_{\text{OMP}}}{T_{\text{ASL-SR-DPT}}}$$

and

$$\text{Speedup}_{\text{LASSO}} = \frac{T_{\text{LASSO}}}{T_{\text{ASL-SR-DPT}}}$$

Where T_{OMP} , T_{LASSO} , and $T_{\text{ASL-SR-DPT}}$ represent the mean CPU solve latency of OMP, LASSO, and the ASL-SR-DPT, respectively. A speedup value greater than 1 indicates that the ASL-SR-DPT solver has lower latency than the corresponding baseline method.

Latency Reduction: The percentage reduction in latency of the ASL-SR-DPT solver compared with a selected baseline is calculated as:

$$\text{Latency Reduction} = \left(1 - \frac{T_{\text{ASL-SR-DPT}}}{T_{\text{baseline}}}\right) \times 100\%$$

Where T_{baseline} represents the mean latency of the baseline method being compared.

Data Analysis Procedure

The gathered simulation metrics are analyzed methodically to extract engineering insights regarding the computational characteristics of the algorithm under the controlled virtualized environment and its potential suitability for future resource-constrained deployment. The analysis is structured strictly around five quantitative performance metrics:

1. Image Quality Metric: Spatial Peak Signal-to-Noise Ratio (PSNR)

PSNR is a logarithmic image-fidelity metric derived from the mean squared error between the reference image and the reconstructed image. Higher PSNR values indicate lower reconstruction error. It is computed in decibels (dB) as:

$$\text{PSNR} = 10 \log_{10} \left(\frac{MAX^2}{MSE} \right)$$

PSNR improvement is calculated as the difference between the reconstructed-image PSNR and the corresponding noisy-image PSNR.

2. Distortion Metric: Spatial Mean Squared Error (MSE)

MSE computes the average squared intensity difference between the original noise-free spatial image (I) and the fully aggregated, reconstructed spatial image ($\hat{I}$):

$$MSE_{\text{spatial}} = \frac{1}{H \times W} \sum_{x=1}^H \sum_{y=1}^W \left(I(x, y) - \hat{I}(x, y) \right)^2$$

where H and W represent the height and width of the spatial image block. A lower MSE indicates closer mathematical proximity to the ground truth.

3. Structural Similarity Metric: Structural Similarity Index (SSIM)

SSIM is a perceptual metric that measures structural fidelity by evaluating luminance, contrast, and structural information between the original noise-free spatial image and the reconstructed image.

4. Computational Latency Metric: Measured Iterative Solve Time

The high-precision measured CPU time required by each solver to process the specified set of image patches. One-time sensing-matrix and pseudoinverse precomputation time is reported separately from iterative solve time. This aligns with your corrected VM methodology.

5. Active-Support Computation Metric

To quantify active-support utilization, the system records the number of active coefficients at each executed solver iteration. The active-support ratio is calculated as:

$$\frac{\sum_{i=1}^{k_{\max}} \left\| z_{\text{active}}^{(i)} \right\|_0}{k_{\max} \times N}$$

, where

$$\left\| z_{\text{active}}^{(i)} \right\|_0$$

denotes the number of active coefficients at iteration i , k_{max} represents the actual number of executed iterations, and $N=64$ for an 8×8 DCT patch. A lower ratio indicates that fewer coefficients participate in gradient calculations and serves as an empirical proxy for reduced coefficient-level computation.